CS 147L · Interview 2 Cheat Sheet

React Native + TypeScript · patterns, gotchas, errors, recipes

The 5 Golden Patterns

In one sentence. If you can type these five from memory, you can build almost any Interview 2 app: **state from data, map to components, add immutably, callback down and up, controlled input.**

1 · State from data

```
// in App: inference gives menu: MenuItem[]
const [menu, setMenu] = useState(menuItems);
// an empty array needs a type:
const [list, setList] = useState<MenuItem[]>([]);
// name + popular state lives in the form (pattern 4)
```

2 · Map data to components

```
// no { after =>, so the JSX is returned
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
// then inside JSX:
<ScrollView>{cards}</ScrollView>
```

2b · The card: typed, destructured props

```
const ItemCard = ({ item }: ItemCardProps) => (
  <View style={styles.card}>
    <Text>{item.name}</Text>
    {item.popular && <Text>Popular</Text>}
  </View>
);
// item.popular is a boolean, so && is safe
```

3 · Add to an array immutably

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]); // new array!
  // also valid: setMenu((prev) => [...prev, item]);
};
```

4a · Parent (App): owns the list

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
};
// pass the function down, no ()
<AddItemForm onAdd={addMenuItem} />
```

4b · Child: owns the text field and switch

```
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  const handleAdd = () => {
    if (name.trim() === "") return;
    onAdd({ name, popular }); // call it UP
    setName("");
    setPopular(false);
  };
  return (
    <View>
      <TextInput value={name} onChangeText={setName} placeholder="Item name" />
      <Switch value={popular} onValueChange={setPopular} />
      <Pressable onPress={handleAdd}><Text>Add</Text></Pressable>
    </View>
  );
};
```

5 · Controlled inputs

```
// value comes from state; the handler calls the setter
<TextInput value={name} onChangeText={setName} />
// same thing: onChangeText={(text) => setName(text)}
<Switch value={popular} onValueChange={setPopular} />
```

Tricky stuff: wrong → right → why

Wrong	Right	Why
`setMenu(menu => {...menu, item})`	`setMenu([...menu, item])` or `setMenu((prev) => [...prev, item])`	Braces right after `=>` start a function body, not an object. And state is an array, so build `[...]`, not `{...}`.
`menu.map((i) => { <ItemCard item={i} /> })`	`menu.map((i) => (<ItemCard key={i.name} item={i} />))`	A block body without `return` returns `undefined`, so nothing renders. Drop the braces (parens around the JSX are fine) or add `return`.
`onPress={addItem(x)}`	`onPress={() => addItem(x)}`	The first calls `addItem` during render. If it sets state, you get "Too many re-renders".
`menu.push(item); setMenu(menu);`	`setMenu([...menu, item])`	Same array object, so React sees no change and skips the re-render. Never mutate state.
`<TextInput onChange={setName} />`	`<TextInput onChangeText={setName} />`	`onChange` passes an event object. `onChangeText` passes the string.
`menu.map((m) => <ItemCard item={m} />)`	`<ItemCard key={m.name} item={m} />`	Every list item needs a unique, stable `key` so React can track it.
`{menu.length && <List />}`	`{menu.length > 0 && <List />}`	When the length is `0`, React renders `0` as raw text outside `<Text>`.
`useState([])`	`useState<MenuItem[]>([])`	An empty array is inferred as `never[]`, so nothing can be added to it.
`() => { name: "Tea" }`	`() => ({ name: "Tea" })`	Without parens, the braces are a body and the function returns `undefined`.
`setState([...menu, item])`	`setMenu([...menu, item])`	Only the setter you declared exists. A generic `setState` gives "Cannot find name".
`setCount(count + 1); console.log(count);`	`const next = count + 1; setCount(next); console.log(next);`	State is a snapshot. The variable updates on the next render, not on the next line.
`style="color: red"`	`style={{ color: "red" }}` or `style={styles.title}`	Styles are JavaScript objects with camelCase keys, not CSS strings.
`<View>Hello</View>`	`<View><Text>Hello</Text></View>`	Every piece of text must be inside `<Text>`. Same for `//` comments in JSX: write `{/* note */}`.
Expecting children side by side	Add `flexDirection: "row"`	React Native defaults to `"column"`. The web defaults to `"row"`.
`onAdd(name, popular)`	`onAdd({ name, popular })`	The prop type says `onAdd` takes one `MenuItem` object, so build that object.

Error decoder: message → meaning → fix

Message (exact text)	What it means	Fix
`Text strings must be rendered within a <Text> component.`	A raw string, `0`, `""`, or a `//` comment sits directly inside a `<View>`.	Wrap it in `<Text>`. Use `{/* */}` for comments. Use `count > 0 &&`.

Message (exact text)	What it means	Fix
`Each child in a list should have a unique "key" prop.`	A `map` result has no `key`, or two keys repeat.	Add `key={item.id}` (or `key={item.name}` if names are unique) to the outer element returned by `map`.
`Too many re-renders. React limits the number of renders to prevent an infinite loop.`	State is set during render, usually `onPress={setCount(count + 1)}`.	Wrap it: `onPress={() => setCount(count + 1)}`.
`Property 'nmae' does not exist on type 'MenuItem'.`	A typo, or a field missing from your type. With `on type 'never'`, your state came from `useState([])`.	Fix the spelling, add the field to the type, or use `useState<MenuItem[]>([])`.
`Type 'string' is not assignable to type 'number'.`	You gave a string where a number was expected, such as `price="5"`.	Pass the right type: `price={5}`, or `Number(text)`.
`Type 'MenuItem' is not assignable to type 'never'.` · `Argument of type 'MenuItem' is not assignable to parameter of type 'never'.`	The array state was created with `useState([])`, so it is `never[]`. With the updater form it may also mention `SetStateAction<never[]>`.	`useState<MenuItem[]>([])`.
`Property 'onAdd' is missing in type '{ item: MenuItem; }' but required in type 'RowProps'.`	You rendered a component without a required prop.	Pass the prop, or mark it optional with `onAdd?:`.
`Parameter 'item' implicitly has an 'any' type.`	A standalone function parameter has no type.	`(item: MenuItem) => ...`
`View is not defined` · `Property 'View' doesn't exist` · `Can't find variable: View` · `Cannot find name 'View'.`	The name was never imported or declared, or it is misspelled.	Add it to the import from `"react-native"` or `"react"`. Check capitals: `TextInput`, not `Textinput`.
`Objects are not valid as a React child (found: object with keys {name, popular}).`	You put a whole object in JSX, such as `<Text>{item}</Text>`.	Render one field: `{item.name}`.
`Cannot read property 'map' of undefined` · `menu.map is not a function`	The value is not an array, often after `setMenu(menu.push(x))` made it a number.	Start with an array and update with `[...menu, x]`.
`Adjacent JSX elements must be wrapped in an enclosing tag.`	A component returns two siblings with no parent.	Wrap them in `<View>` or `<>...</>`.
`Maximum update depth exceeded.`	An effect sets state on every render, usually with no dependency array.	Add `[]` or `[dep]`, and only set state when needed.

Arrow functions

In one sentence. An arrow function is a **value** you can store, pass as a prop, or hand to `map`: `(params) => result`.

Four equivalent forms

```
function double(x: number) { return x * 2; }
const double2 = (x: number) => { return x * 2; };
const double3 = (x: number) => x * 2;
const double4 = (x: number) => (x * 2);
```

- **No `{` right after `=>`** means an expression body: the value is returned automatically. Parens around multi-line JSX are only grouping.
- **A `{` right after `=>`** starts a block body (a function body): you must write `return`.

Returning an object: parens are required

```
const makeItem = (name: string) => ({
  name,
  popular: false,
});
```

Pass vs call

You write	What happens
`onPress={save}`	Passes the function. React calls it on press.
`onPress={save()}`	Calls it **now**, during render, and passes the result. If it sets state, you get "Too many re-renders".
`onPress={() => save(item)}`	Passes a new function that calls `save(item)` later. Use this when you need arguments.

map & lists

In one sentence. `map` turns an array of data into an array of JSX. Store it in a variable, then render it with `{cards}`.

One-liner (no { after =>, so it returns)

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
```

Block body (braces need return)

```
const rows = menu.map((item, index) => {
  const label = `${index + 1}. ${item.name}`;
  return <Text key={item.name}>{label}</Text>;
});
```

- **Render it:** `<ScrollView>{cards}</ScrollView>`, or inline `{menu.map(...)}` inside JSX.
- **`key`** goes on the outermost element that `map` returns. Use a unique id if the data has one (`key={item.id}`). `key={item.name}` is fine only if names are unique. Avoid the index as your default.
- The second parameter is the index: `(item, index) => ...`.

Cousins

```
const hot = menu.filter((m) => m.popular);   // new array
const tea = menu.find((m) => m.name === "Tea"); // item | undefined
const has = menu.some((m) => m.popular);    // boolean
```

Spread & immutable updates

In one sentence. Never change state in place. **Build a new array or object** and pass it to the setter.

Goal	Recipe
Add to end	`setMenu([...menu, item])`
Add to start	`setMenu([item, ...menu])`
Remove	`setMenu(menu.filter((m) => m.name !== target))`
Update / toggle one	`map` + `{ ...m, popular: !m.popular }` (below)
Change an object field	`setUser({ ...user, name: "Ada" })`
Count (no extra state)	`const popularCount = menu.filter((m) => m.popular).length;`

```
const toggle = (target: string) => {
  setMenu(
    menu.map((m) =>
      m.name === target
        ? { ...m, popular: !m.popular }
        : m
    )
  );
};
```

- **Never on state:** `push`, `splice`, `pop`, `sort`, or `item.popular = true`. They change the existing array, React sees the same array, and nothing re-renders. Also, `push` returns the new length, not the array.
- `[...]` makes an array, `{...}` makes an object. Match the shape of your state.

- Destructuring: `const { name, popular } = item;`

useState

> **In one sentence.** State is memory that survives renders. **Calling its setter is the only way to change it**, and that triggers a re-render.

```
const [menu, setMenu] = useState(menuItems);
//     value  setter           initial value
```

Direct vs updater (both valid)

```
setCount(count + 1);          // direct: simplest default
setCount((prev) => prev + 1); // updater: uses latest value
```

- **Direct form** is the easy default for event handlers.
- **Updater form** is safer when the new value depends on the old one inside timers, or when you update several times in a row.
- **Snapshot rule:** after `setCount(5)`, the variable `count` keeps its old value until the next render.

Inference does the typing

`useState("")`	`string`
`useState(false)`	`boolean`
`useState(menuItems)`	`MenuItem[]`
`useState<MenuItem[]>([])`	empty array **needs** the type

- Call the setter you declared (`setMenu`), not a generic `setState`.
- Hooks go at the top level of a component: not inside `if`, loops, or handlers.

Callbacks: data down, events up

> **In one sentence.** Define the handler in the **parent where state lives**, pass it as a prop, and let the child call it inside an arrow function.

```
// parent: handler lives with the state
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
};
<AddItemForm onAdd={addMenuItem} />

// child: build the object the type expects
type AddItemFormProps = {
  onAdd: (item: MenuItem) => void;
};
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  return (
    <Pressable onPress={() => onAdd({ name, popular })}>
      <Text>Add</Text>
    </Pressable>
  );
};
```

- Pass the function **without** `()`: `onAdd={addMenuItem}`.
- `{ name, popular }` is shorthand for `{ name: name, popular: popular }`.
- Lift state up: put it in the closest parent that every user of it shares.

Inputs

> **In one sentence.** Inputs are **controlled**: `value` comes from state, and the change handler calls the setter.

Component	Key props	Handler receives
`TextInput`	`value`, `onChangeText`, `placeholder`	the new `string`
`Switch`	`value`, `onValueChange`	the new `boolean`
`Pressable`	`onPress`, `style`, children such as `<Text>`	a press event
`Button`	`title` and `onPress` (the docs mark both as required). No `style` prop.	a press event

```
<View style={styles.form}>
  <TextInput
    style={styles.input}
    value={name}
    onChangeText={setName}
    placeholder="Item name"
  />
  <Switch value={popular} onValueChange={setPopular} />
  <Button title="Add" onPress={handleAdd} />
</View>
```

Add, then clear the input

```
const handleAdd = () => {
  if (name.trim() === "") return; // skip empty names
  onAdd({ name, popular });
  setName("");
  setPopular(false);
};
```

Pressable that dims while pressed

```
<Pressable
  onPress={handleAdd}
  style={({ pressed }) => [styles.btn, pressed && { opacity: 0.6 }]}
>
  <Text>Add</Text>
</Pressable>
```

- `onChangeText={setName}` and `onChangeText={(text) => setName(text)}` do the same thing.
- Use `onChangeText`, not `onChange`. `onChange` hands you an event object instead of the string.
- `onPress={setPopular}` is **not** like `onChangeText={setName}`: `onPress` passes a press event object, not a value. Write `onPress={() => setPopular(true)}`.
- Give TextInput a `borderWidth` and `padding` so you can see it.

Conditional rendering

> **In one sentence.** Use plain JavaScript to choose what to show: an **if/else variable**, a **ternary**, or **`&&` with a true boolean**.

```
// 1. if/else into a variable (empty state)
let content = <Text>No items yet</Text>;
if (menu.length > 0) {
  content = <ScrollView>{cards}</ScrollView>;
}

// 2. ternary: this OR that (inside JSX)
{menu.length === 0 ? <Text>No items yet</Text> : cards}

// 3. &&: this OR nothing (left side boolean!)
{menu.length > 0 && <Text>{menu.length} items</Text>}
```

- **Never put a number on the left of `&&`.** If `menu.length` is `0`, React renders the `0`, and React Native complains that text must be inside `<Text>`. Write `> 0`.
- `false`, `null`, and `undefined` render nothing.
- Conditional style: `style={[styles.card, item.popular && styles.popularCard]}`. A `false` entry in a style array is skipped.

TypeScript

In one sentence. Write it in JavaScript first and let inference work. **Add types where TypeScript cannot guess**: props, function parameters, and empty arrays.

```
type MenuItem = {
  name: string;
  popular: boolean;
  price?: number;               // optional
};
interface Song { title: string; liked: boolean }
type Size = "small" | "medium" | "large"; // union
type AddFn = (item: MenuItem) => void;     // function type
type AddItemFormProps = { onAdd: AddFn };
```

Thing	Annotation
Primitives	`string`, `number`, `boolean`
Array	`MenuItem[]` (same as `Array<MenuItem>`)
Empty state	`useState<MenuItem[]>([])`
Maybe missing	`useState<MenuItem | null>(null)`
Function parameter	`(item: MenuItem) => { ... }`
Callback prop	`onAdd: (item: MenuItem) => void`
Props	`({ item }: ItemCardProps)`
Text handler	`(text: string) => void`

- **`type` vs `interface`:** both describe object shapes. Only `type` can name unions. Pick one and stay consistent.
- `void` means the function returns nothing useful. `?` means the field may be missing.
- Read red squiggles left to right: "Type **A** is not assignable to type **B**" means you gave A where B was expected. Avoid `any`.

Layout: Flexbox & StyleSheet

In one sentence. In React Native, **`flexDirection` defaults to `"column"`**. `justifyContent` works along the main axis and `alignItems` works across it.

- **`flex: 1`** fills the available space along the main axis. Its parent needs a size too, or it collapses to 0.
- `alignItems` defaults to `"stretch"`. `flexWrap: "wrap"` on the container lets children flow onto new lines.
- `gap`, `rowGap`, `columnGap` space the children. Sizes are unitless numbers, never `"px"`. Names are camelCase.

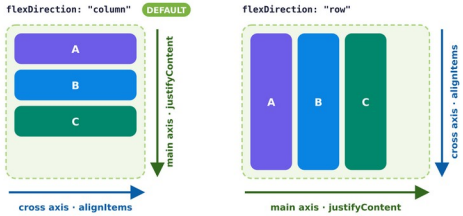
```
const styles = StyleSheet.create({
  container: { flex: 1, padding: 16, gap: 12 },
  row: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
  },
  grid: { flexDirection: "row", flexWrap: "wrap" },
  hot: { backgroundColor: "#fff6e6" },
});
// style array: later entries win, false is skipped
<View style={[styles.row, isHot && styles.hot]} />
```

ScrollView vs FlatList

ScrollView	FlatList
Renders all children at once. Great for short lists: `{cards}`.	Renders lazily. Better for long lists.
Padding and alignment go in `contentContainerStyle`.	Props: `data`, `renderItem`, `keyExtractor`.

```
<FlatList
  data={menu}
  keyExtractor={(m) => m.name}
  renderItem={({ item }) => <ItemCard item={item} />}
/>
```

- Do not put a FlatList inside a ScrollView that scrolls the same direction.



useEffect, timers, Promises

In one sentence. An effect runs **after** render to sync with something outside React, such as a timer or a data request. The dependency array controls when.

You write	The effect runs
`useEffect(fn)`	after every render
`useEffect(fn, [])`	once, after the first render
`useEffect(fn, [a, b])`	after the first render and whenever `a` or `b` changes

- **Cleanup:** the function you return runs right before the effect runs next time, and when the component unmounts.

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds((prev) => prev + 1); // updater!
  }, 1000);
  return () => clearInterval(id);   // cleanup
}, []);
```

- **Why the updater here:** with `[]`, the interval callback remembers the `seconds` from the first render (a stale closure). `setSeconds(seconds + 1)` would get stuck at 1.
- Rules of hooks: call `useEffect` and `useState` at the top level, never inside `if`, loops, or handlers, and never after an early `return`.

Promises: attach callbacks with .then

```
// fetchData() returns a Promise
fetchData()
  .then((data) => setItems(data))
  .catch((err) => console.log(err));
```

App skeleton: type it from memory

In one sentence. This is the 2-minute scaffold: **imports, type, child with typed props, App with state, map, callback, StyleSheet.** Practice typing it until it feels automatic.

```
import { useState } from "react";
import {
  View,
  Text,
  TextInput,
  Pressable,
  ScrollView,
  Switch,
  StyleSheet,
} from "react-native";
```

```tsx
// 1. Type + starting data
type MenuItem = {
  name: string;
  popular: boolean;
};

const menuItems: MenuItem[] = [
  { name: "Tea", popular: true },
  { name: "Toast", popular: false },
];

// 2. Child that displays one item
type ItemCardProps = { item: MenuItem };

const ItemCard = ({ item }: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text>{item.name}</Text>
      {item.popular && <Text>Popular</Text>}
    </View>
  );
};

// 3. Child that reports a new item UP
type AddItemFormProps = {
  onAdd: (item: MenuItem) => void;
};

const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);

  const handleAdd = () => {
    if (name.trim() === "") return;
    onAdd({ name, popular });
    setName("");
    setPopular(false);
  };

  return (
    <View style={styles.form}>
      <TextInput
        style={styles.input}
        value={name}
        onChangeText={setName}
        placeholder="Item name"
      />
      <View style={styles.row}>
        <Text>Popular?</Text>
        <Switch value={popular} onValueChange={setPopular} />
      </View>
      <Pressable style={styles.button} onPress={handleAdd}>
        <Text style={styles.buttonText}>Add</Text>
      </Pressable>
    </View>
  );
};

// 4. App owns the state + the callback
export default function App() {
  const [menu, setMenu] = useState(menuItems);

  const addMenuItem = (item: MenuItem) => {
    setMenu([...menu, item]);
  };

  const cards = menu.map((item) => (
    <ItemCard key={item.name} item={item} />
  ));

  return (
    <View style={styles.container}>
      <AddItemForm onAdd={addMenuItem} />
      <ScrollView>{cards}</ScrollView>
    </View>
  );
}

// 5. Styles
const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, paddingTop: 60, gap: 12 },
  form: { gap: 8 },
  row: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
  },
  input: { borderWidth: 1, borderColor: "#ccc", borderRadius: 8, padding: 10 },
  button: {
    backgroundColor: "#6c5ce7",
    padding: 12,
    borderRadius: 8,
    alignItems: "center",
  },
  buttonText: { color: "white", fontWeight: "bold" },
  card: {
    padding: 12,
    marginBottom: 8,
    borderRadius: 8,
    backgroundColor: "#f0eeff",
  },
});
```

- Build in this order: type and data, static JSX, state and `map`, then the form and callback.
- Run it after every step so each error points at a few new lines.